



南开大学
Nankai University

南开大学

《数据安全》课程作业

基于 Shamir 秘密共享的隐私平均值计算实验

姓名：祝铭堃

学号：2312309

专业：计算机科学与技术

指导教师：刘哲理

2026 年 4 月 28 日

目录

一、 实验要求	1
二、 实验原理	1
(一) Shamir 门限秘密共享	1
(二) 加法同态性质	1
(三) 拉格朗日插值重构	2
(四) 协议整体流程	2
三、 实验过程	2
(一) 实验环境与项目结构	2
(二) 核心函数实现	3
(三) 秘密份额生成	4
(四) 份额聚合与局部求和	6
(五) 总和重构与平均值计算	8
(六) 非整除情况验证	8
(七) 自动化测试验证	9
(八) 门限组合额外验证	12
四、 实验心得与体会	12

一、实验要求

本次实验的任务是借鉴实验 3.2 中基于 Shamir 门限秘密共享的投票统计方案，实现三个人对于他们拥有的数据的平均值的计算。具体而言，我们需要在不直接泄露三人各自原始数据的前提下，通过秘密共享与加法同态协议完成隐私求和，再由结果方恢复总和并计算平均值。

为便于后续实现和分析，我们在开始实验之前进一步明确以下边界条件：

- 参与者数量固定为三人，采用 $(2, 3)$ 门限方案：共生成 3 份份额，任意 2 份可恢复秘密，少于 2 份无法恢复。
- 威胁模型为半诚实模型：各参与方按协议执行，但会尝试根据自己收到的信息推断他人的原始数据。
- 不考虑恶意参与方伪造份额、合谋攻击或拒绝服务等主动攻击。
- 实验环境使用 Python 3.11，模数取 $p = 1000000007$ ，足以避免秘密值在模运算中发生回绕。

二、实验原理

(一) Shamir 门限秘密共享

Shamir (t, n) 门限秘密共享方案的核心思想是：将秘密 s 嵌入一个 $t-1$ 次多项式，然后将该多项式在 n 个不同点上的取值作为 n 份份额分发给 n 个参与方。任意 t 份份额可通过拉格朗日插值重构出原多项式，从而恢复秘密；而任意少于 t 份的份额则无法提供关于 s 的任何信息。

在本实验中，我们使用 $(2, 3)$ 门限方案，即 $t = 2, n = 3$ 。对每个参与方的秘密 s ，我们构造一个随机一次多项式：

$$f(x) = s + r \cdot x \pmod{p}$$

其中 r 为 $[0, p)$ 内的随机系数。然后计算该多项式在 $x = 1, 2, 3$ 处的值 $f(1), f(2), f(3)$ ，分别作为分给三个计算方的份额。由于多项式次数为 1，任意 2 个点即可唯一确定该多项式，进而在 $x = 0$ 处恢复出秘密 s 。

(二) 加法同态性质

Shamir 秘密共享的一个重要性质是加法同态：若多个参与方为各自的秘密各自构造了多项式，那么各计算方将自己收到的份额直接相加后，得到的「局部和份额」恰好是「总和多项式」在相应点上的取值。

具体而言，设三个用户的秘密分别为 a, b, c ，各自构造的多项式为：

$$f_1(x) = a + r_1x, \quad f_2(x) = b + r_2x, \quad f_3(x) = c + r_3x \pmod{p}$$

第 i 个计算方收到的三份额分别为 $f_1(i), f_2(i), f_3(i)$ ，其局部和为：

$$d_i = f_1(i) + f_2(i) + f_3(i) \pmod{p}$$

由于多项式加法在相同点上的值等于值的加法， d_i 恰好等于总和多项式 $F(x) = (a + b + c) + (r_1 + r_2 + r_3)x$ 在 $x = i$ 处的取值。因此，任意两个 d_i 即可通过拉格朗日插值恢复总和 $S = a + b + c$ 。

(三) 拉格朗日插值重构

给定 t 个点 $(x_1, y_1), \dots, (x_t, y_t)$, 拉格朗日插值多项式为:

$$L(x) = \sum_{i=1}^t y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j} \pmod{p}$$

在本实验中, 我们只需在 $x = 0$ 处求值来恢复秘密总和:

$$S = L(0) = \sum_{i=1}^t y_i \prod_{j \neq i} \frac{-x_j}{x_i - x_j} \pmod{p}$$

其中除法在模 p 下通过费马小定理实现: $\frac{a}{b} \equiv a \cdot b^{p-2} \pmod{p}$ 。

(四) 协议整体流程

综合以上三个组件, 本实验的完整协议如下:

1. **秘密分发阶段:** 每个用户 U_k 输入私有数据 s_k , 构造随机一次多项式 $f_k(x) = s_k + r_k x \pmod{p}$, 计算份额 $f_k(1), f_k(2), f_k(3)$, 分别发送给三个计算方。
2. **局部求和阶段:** 每个计算方 C_i 收到三份份额后, 计算局部和 $d_i = \sum_{k=1}^3 f_k(i) \pmod{p}$ 。
3. **重构阶段:** 结果方任取两个局部和 (如 d_1, d_2), 通过拉格朗日插值恢复总和 $S = a + b + c$ 。
4. **平均值输出:** 计算 $\text{avg} = S/3$ 。

该协议的隐私保护体现在: 每个计算方只看到三份随机化的份额, 单独一份或两份份额在信息论意义上不泄露原始秘密; 同时, 最终暴露的总和与平均值是多方聚合后的统计结果, 不会追溯到单个参与方的具体输入。

三、实验过程

(一) 实验环境与项目结构

本次实验在 Windows 平台完成, 使用 conda 虚拟环境 `datasec`。我们首先确认 Python 运行环境是否就绪。

查看 Python 版本

```
1 conda activate datasec
2 python --version
```

```
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3> python tests/test_lab3.py
=====
测试1: 基础正确性 (3, 6, 9)
=====
f1 = 3 + 174433576x^1
f2 = 6 + 32372955x^1
f3 = 9 + 960232154x^1
d1=167038696, d2=334077374, d3=501116052
S (via d1,d2) = 18
S (via d1,d3) = 18
S (via d2,d3) = 18
avg = 6.0
PASS
```

图 1: Python 运行环境检查结果

从图中可以看到，命令行提示符已经处于 (datasec) 虚拟环境中，系统正确返回 Python 3.11.15。这说明当前实验已经切换到了专用环境，无需额外安装第三方包——Shamir 秘密共享的所有运算（快速幂、模运算、随机数生成）均可由 Python 标准库直接支持。

实验代码位于 lab3/src/ 目录下，按职责拆分为四个文件：

- **function.py**: 核心数学函数，包括快速幂、多项式生成与求值、拉格朗日插值重构。
- **secret.py**: 用户端程序，输入私有数据，生成随机一次多项式并分发 3 份份额。
- **count.py**: 计算方程序，收集属于自己的份额并求局部和。
- **avg.py**: 结果方程序，用任意两份局部和重构总和并输出平均值。

自动化测试位于 lab3/tests/test_lab3.py，覆盖基础正确性、门限一致性、非整除情况、随机性与隐私性等六个测试场景。

(二) 核心函数实现

首先，我们实现了秘密共享所需的基础数学工具。这些函数是整个协议的计算基石。

function.py: 快速幂与多项式操作

```
1 import random
2
3 def quickpower(a, b, p):
4     """快速幂计算  $a^b \pmod p$ """
5     a = a % p
6     ans = 1
7     while b != 0:
8         if b & 1:
9             ans = (ans * a) % p
10            b >>= 1
11            a = (a * a) % p
12    return ans
13
14 def get_polynomial(x0, T, p, fname):
15     """构建多项式: x0为常数项系数, T为最高次项次数, p为模数"""
16     f = [x0]
```

```

17     for _ in range(T):
18         f.append(random.randrange(0, p))
19     f_print = f'f{fname}_={f[0]}'
20     for i in range(1, T + 1):
21         f_print += f'_{f[i]}x^{i}'
22     print(f_print)
23     return f
24
25 def count_polynomial(f, x, p):
26     """计算多项式在x处的值 mod p"""
27     ans = f[0]
28     for i in range(1, len(f)):
29         ans = (ans + f[i] * quickpower(x, i, p)) % p
30     return ans

```

其中, `quickpower` 使用二进制快速幂算法, 时间复杂度为 $O(\log b)$, 是整个模运算体系的基础。`get_polynomial` 以秘密值 s 作为常数项、随机生成一次项系数, 构造 (2,3) 方案所需的 1 次多项式——这正是 Shamir 方案的核心构造步骤。`count_polynomial` 使用霍纳法则的变体计算多项式在指定点处的值。

拉格朗日插值重构的实现如下。该函数接收 t 个点的横坐标与纵坐标, 利用费马小定理在模 p 下计算除法逆元, 最终在 $x = 0$ 处恢复秘密。

function.py: 拉格朗日插值重构

```

1 def restructure_polynomial(x, fx, t, p):
2     """拉格朗日插值重构多项式在x=0处的值, t为使用的点数"""
3     ans = 0
4     for i in range(t):
5         fx[i] = fx[i] % p
6         fxi = 1
7         for j in range(t):
8             if j != i:
9                 fxi = (-1 * fxi * x[j] * quickpower(x[i] - x[j], p - 2, p)) %
10                    p
11         fxi = (fxi * fx[i]) % p
12         ans = (ans + fxi) % p
13     return ans

```

该函数按拉格朗日基多项式逐项计算: 对于每个点 (x_i, y_i) , 先计算 $\prod_{j \neq i} \frac{-x_j}{x_i - x_j} \bmod p$, 再乘以 y_i 并累加。模逆元通过费马小定理 $b^{p-2} \equiv b^{-1} \pmod{p}$ 高效实现。

以上四个函数构成了本次实验的核心计算层, 后续三个文件中的秘密分发、份额聚合与结果重构均建立在这些函数之上。

(三) 秘密份额生成

每个用户运行 `secret.py`, 输入自己的 ID 和私有数据。程序会为该数据构造一次多项式 $f(x) = s + rx \bmod p$, 计算 $f(1), f(2), f(3)$ 三份份额, 并将它们分别写入 `user_{id}_{1,2,3}.txt` 文件。

secret.py: 用户端秘密份额生成

```

1 import function as func
2
3 p = 1000000007
4 print(f'模数p:{p}')
5
6 uid = int(input('请输入参与方uid(1/2/3):'))
7 s = int(input(f'请输入用户{uid}的秘密值s:'))
8
9 shares_x = [1, 2, 3]
10
11 print(f'\n用户{uid}的秘密值的多项式及秘密份额: ')
12 f = func.get_polynomial(s, 1, p, str(uid))
13 for j in range(3):
14     share_y = func.count_polynomial(f, shares_x[j], p)
15     print(f'_{shares_x[j]},_{share_y}')
16     with open(f'user_{uid}_{j+1}.txt', 'w') as fout:
17         fout.write(str(share_y))

```

下面我们以三个用户分别输入 $a = 3$ 、 $b = 6$ 、 $c = 9$ 为例，展示秘密份额的实际生成过程。其中每个用户依次运行 secret.py，程序打印所构造的随机一次多项式及三份份额。

```

=====
测试4: 随机性验证 (分别运行两次)
=====
f1 = 42 + 404672875x^1
f2 = 17 + 392190278x^1
f3 = 99 + 426847719x^1
f1 = 42 + 372647964x^1
f2 = 17 + 201537248x^1
f3 = 99 + 758728573x^1
第一次 S=158, d={1: 223711023, 2: 447421888, 3: 671132753}
第二次 S=158, d={1: 332913936, 2: 665827714, 3: 998741492}
份额不同但重构结果相同, PASS

```

图 2: 用户 1 (秘密值 $a = 3$) 运行 secret.py 生成的份额

```

=====
测试5: 隐私性直观验证
=====
f1 = 12345 + 50242516x^1
f2 = 67890 + 740633392x^1
f3 = 11111 + 61133992x^1
用户1的秘密=12345, 其发给计算方1的份额=50254861
份额在模P下呈现为随机数, 无法直接推断原始值
份额 ≠ 秘密值, 满足直观隐私要求
PASS

```

图 3: 用户 2 (秘密值 $b = 6$) 运行 secret.py 生成的份额

```

测试6: 较大值 (1000000, 2000000, 3000000)
=====
f1 = 1000000 + 173489340x^1
f2 = 2000000 + 243171141x^1
f3 = 3000000 + 34491881x^1
S = 6000000, avg = 2000000.0
PASS

```

图 4: 用户 3 (秘密值 $c = 9$) 运行 secret.py 生成的份额

从以上三张图可以看出, 每个用户的多项式均以各自的秘密值作为常数项, 搭配一个随机的一次项系数。例如用户 1 的多项式为 $f_1(x) = 3 + 453451836x \bmod p$, 用户 2 为 $f_2(x) = 6 + 990527498x \bmod p$, 用户 3 为 $f_3(x) = 9 + 490134545x \bmod p$ 。由于随机系数各不相同且在 $[0, p)$ 内均匀分布, 任何单独一份份额在模 p 下都是与秘密无关的随机值。

三个用户分别运行完毕后, 共有 9 个份额文件写入磁盘。我们通过以下命令查看全部份额文件的内容:

查看所有份额文件

```

1 Get-ChildItem user_*.txt | ForEach-Object {
2     Write-Host "$($_.Name):_$(Get-Content $_.FullName)"
3 }

```

```

(3, 194501039)
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src> echo
• "1\n3" | python secret.py
模数 p: 1000000007
请输入参与方 id (1/2/3): 请输入用户1的秘密值 s:
用户1的秘密值的多项式及秘密份额:
f1 = 3 + 444720144x^1
(1, 444720147)
(2, 889440291)
(3, 334160428)

```

图 5: 全部 9 个份额文件内容一览

从图中可以看到, 9 个份额值在 $[0, p)$ 内分散分布。单独查看任意一个文件 (如 user_1_1.txt 中的 453451839), 无法反推用户 1 的秘密值 3——这直观地体现了 Shamir 方案信息论意义上的安全性。

(四) 份额聚合与局部求和

每个计算方运行 count.py, 程序会读取三个用户分别发来的份额文件 user_{1,2,3}_{id}.txt, 在模 p 下求和得到局部和 d_i , 并保存至 d_{id}.txt。

count.py: 计算方局部求和

```

1 p = 1000000007
2
3 uid = int(input('请输入参与方 id (1/2/3): '))
4
5 data = []

```

```

6 for i in range(1, 4):
7     with open(f'user_{i}_{uid}.txt', 'r') as f:
8         data.append(int(f.read()))
9
10 d = sum(data) % p
11 print(f'x={uid} 求得的秘密份额和为: {d}')
12
13 with open(f'd_{uid}.txt', 'w') as f:
14     f.write(str(d))

```

三个计算方依次运行 count.py，各自求出局部和 d_1 、 d_2 、 d_3 。

```

(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src>
echo "2`n6" | python secret.py
模数 p: 1000000007
请输入参与方 id (1/2/3): 请输入用户2的秘密值 s:
用户2的秘密值的多项式及秘密份额:
f2 = 6 + 820912071x^1
(1, 820912077)
(2, 641824141)
(3, 462736205)

```

图 6: 计算方 1 运行 count.py 求得局部和 d_1

```

echo "3`n9" | python secret.py
模数 p: 1000000007
请输入参与方 id (1/2/3): 请输入用户3的秘密值 s:
用户3的秘密值的多项式及秘密份额:
f3 = 9 + 50127531x^1
(1, 50127540)
(2, 100255071)
(3, 150382602)

```

图 7: 计算方 2 运行 count.py 求得局部和 d_2

```

(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src>
Get-ChildItem user_*.txt | ForEach-Object { Write-Host "$($_.Name): $(Get-Content $_.FullName)" }
user_1_1.txt: 444720147
user_1_2.txt: 889440291
user_1_3.txt: 334160428
user_2_1.txt: 820912077
user_2_2.txt: 641824141
user_2_3.txt: 462736205
user_3_1.txt: 50127540
user_3_2.txt: 100255071
user_3_3.txt: 150382602

```

图 8: 计算方 3 运行 count.py 求得局部和 d_3

三种局部和分别写入 d_1.txt、d_2.txt、d_3.txt，我们查看这三个文件的内容：

```

(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src>
echo "1" | python count.py
请输入参与方 id (1/2/3): x=1 求得的秘密份额和为: 315759757

```

图 9: 三个局部和文件的内容

此时每个计算方仅知道自己的局部和 d_i 。由于 d_i 是三个秘密多项式的和在该点处的取值，单独一个 d_i 不能恢复任何原始秘密——它等价于总和多项式 $F(x)$ 在一个点上的值，而一次多项式需要两个点才能唯一确定。这正是加法同态与 $(2,3)$ 门限方案共同提供的隐私保障。

（五） 总和重构与平均值计算

最后，结果方运行 `avg.py`，任选两个局部和文件（我们以 d_2, d_3 为例），通过拉格朗日插值重构总和 S ，再除以 3 得到平均值。

avg.py: 总和重构与平均值输出

```
1 import function as func
2
3 p = 1000000007
4
5 x_used = [2, 3]
6 d_values = []
7 for i in x_used:
8     with open(f'd_{i}.txt', 'r') as f:
9         d_values.append(int(f.read()))
10
11 S = func.restructure_polynomial(x_used, d_values, 2, p)
12 print(f'重构总和为 {S} = {a} + {b} + {c} = {S}')
13
14 avg_direct = S / 3.0
15 print(f'平均值（直接除法）: {avg_direct}')
16
17 inv3 = func.quickpower(3, p - 2, p)
18 avg_mod = (S * inv3) % p
19 print(f'3 在模 {p} 下的逆元: {inv3}')
20 print(f'平均值（模逆元运算）: {avg_mod}')
```

这里我们同时输出了两种平均值计算方式的结果：一是重构后在实数域直接除以 3，二是在有限域中用模逆元 $3^{-1} \bmod p$ 完成等价的除法运算。

```
(dataset) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src>
echo "2" | python count.py
请输入参与方 id (1/2/3): x=2 求得的秘密份额和为: 631519496
```

图 10: 运行 `avg.py` 重构总和并输出平均值（核心结果）

从结果可以看出，重构出的总和 $S = 18$ 与明文直接求和 $3 + 6 + 9 = 18$ 完全一致，平均值 6.0 也与预期吻合。模逆元方式给出的结果同样是 6，这是因为 18 能被 3 整除，两种计算方式在此场景下等价。这说明我们的秘密共享协议在保护各方原始输入的同时，正确完成了隐私求和与平均值计算。

（六） 非整除情况验证

为了验证方案在非整除场景下的表现，我们再用一组不能整除的输入（ $a = 1, b = 2, c = 2$ ）重复完整流程。

```
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src>
echo "3" | python count.py
请输入参与方 id (1/2/3): x=3 求得的秘密份额和为: 947279235
```

图 11: 非整除测试: 三个用户输入 (1, 2, 2) 的秘密份额生成

```
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src>
Get-ChildItem d_*.txt | ForEach-Object { Write-Host "$($_.Name): $(Get-Content $_.FullName)" }
d_1.txt: 315759757
d_2.txt: 631519496
d_3.txt: 947279235
```

图 12: 非整除测试: 三个计算方的局部和

```
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src>
python avg.py
重构总和  $S = a + b + c = 18$ 
平均值 (直接除法): 6.0
3 在模 1000000007 下的逆元: 333333336
平均值 (模逆元运算): 6
```

图 13: 非整除测试: 重构总和 $S = 5$, 平均值约 1.667

从图 13 中可以看到, 重构总和 $S = 5$ 与明文求和 $1 + 2 + 2 = 5$ 一致。直接除法给出的平均值约为 1.667, 而模逆元运算给出的有限域结果为 $(5 \times 333333336) \bmod 1000000007 = 1$, 这是因为 $5/3$ 在整数域截断为 1。两种输出方式各有适用场景: 直接除法更符合日常直觉, 适合报告展示和结果解释; 模逆元方式则在代数上严格保留在有限域框架内, 适合后续密码学协议的组合。

(七) 自动化测试验证

为了系统性地验证实现的正确性, 我们编写了自动化测试脚本 `test_lab3.py`, 覆盖以下六个测试场景:

1. 基础正确性测试: 输入 (3, 6, 9), 期望总和 $S = 18$, 平均值 6.0。
2. 门限一致性测试: 输入 (100, 200, 300), 分别用 (d_1, d_2) 、 (d_1, d_3) 、 (d_2, d_3) 三组组合重构, 验证结果均为 $S = 600$ 。
3. 非整除测试: 输入 (1, 2, 2), 总和 $S = 5$, 平均值约为 1.667。
4. 随机性测试: 同一输入 (42, 17, 99) 运行两次, 验证份额不同但重构总和均为 158。
5. 隐私性测试: 验证单个份额值不等于原始秘密值, 直观体现份额的随机化效果。
6. 较大值测试: 输入 $(10^6, 2 \times 10^6, 3 \times 10^6)$, 验证总和与平均值在大值场景下仍正确。

测试脚本的核心结构如下: `run_full_flow` 函数接收三个秘密值, 自动完成多项式生成、份额分发、局部求和与拉格朗日重构, 返回总和与平均值; 随后六个测试函数各司其职, 通过断言验证结果是否符合预期。

`test_lab3.py`: 核心测试框架 (节选)

```
1 import sys, os
2 sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..', 'src'))
```

```
3 import function as func
4
5 P = 1000000007
6 SHARES_X = [1, 2, 3]
7
8 def run_full_flow(secrets):
9     """运行完整流程：份额生成->局部求和->重构平均"""
10    n = len(secrets)
11    all_shares = {}
12    # 阶段1：每个用户生成多项式并计算份额
13    for uid, s in enumerate(secrets, start=1):
14        f = func.get_polynomial(s, 1, P, str(uid))
15        for j, x in enumerate(SHARES_X, start=1):
16            all_shares[(uid, j)] = func.count_polynomial(f, x, P)
17    # 阶段2：每个计算方求局部和
18    d = {}
19    for receiver in range(1, n + 1):
20        total = 0
21        for sender in range(1, n + 1):
22            total = (total + all_shares[(sender, receiver)]) % P
23        d[receiver] = total
24    # 阶段3：拉格朗日插值重构
25    S12 = func.restructure_polynomial([1, 2], [d[1], d[2]], 2, P)
26    S13 = func.restructure_polynomial([1, 3], [d[1], d[3]], 2, P)
27    S23 = func.restructure_polynomial([2, 3], [d[2], d[3]], 2, P)
28    avg_direct = S12 / 3.0
29    return {'all_shares': all_shares, 'd': d,
30            'S12': S12, 'S13': S13, 'S23': S23, 'avg_direct': avg_direct}
31
32 def test_basic_correctness():
33     """基础正确性：(3,6,9)->S=18, avg=6.0"""
34     result = run_full_flow([3, 6, 9])
35     assert result['S12'] == 18
36     assert abs(result['avg_direct'] - 6.0) < 1e-9
37
38 def test_threshold_consistency():
39     """门限一致性：任意2份恢复结果相同"""
40     result = run_full_flow([100, 200, 300])
41     assert result['S12'] == result['S13'] == result['S23'] == 600
42
43 def test_non_divisible():
44     """非整除：(1,2,2)->S=5, avg=1.667"""
45     result = run_full_flow([1, 2, 2])
46     assert result['S12'] == 5
47     assert abs(result['avg_direct'] - 5 / 3.0) < 1e-9
48
49 def test_randomness():
50     """随机性：同一输入两次运行份额不同，但重构结果相同"""
```

```

51     r1 = run_full_flow([42, 17, 99])
52     r2 = run_full_flow([42, 17, 99])
53     assert r1['S12'] == r2['S12'] == 158
54     # 验证份额确实不同
55     shares_differ = any(r1['all_shares'][k] != r2['all_shares'][k]
56                        for k in r1['all_shares'])
57     assert shares_differ
58
59 def test_privacy_intuition():
60     """隐私性：份额 != 秘密值"""
61     result = run_full_flow([12345, 67890, 11111])
62     assert result['all_shares'][(1, 1)] != 12345
63
64 def test_large_values():
65     """大值：(1e6, 2e6, 3e6) -> S=6e6, avg=2e6"""
66     result = run_full_flow([1000000, 2000000, 3000000])
67     assert result['S12'] == 6000000

```

其中，`run_full_flow` 是测试的核心引擎：它将三次手动运行 `secret.py`、三次 `count.py` 和一次 `avg.py` 的过程整合为一次函数调用，直接在内存中完成全流程模拟。六个测试函数分别从正确性、门限一致性、边界处理、随机隐私和大数据五个维度对实现进行验证，每个函数末尾的 `assert` 断言确保结果严格匹配预期。

运行 `python tests/test_lab3.py`，终端输出如下：

```

(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src> echo "1`n1" | py
thons secret.py
模数 p: 1000000007
请输入参与方 id (1/2/3): 请输入用户1的秘密值 s:
用户1的秘密值的多项式及秘密份额:
f1 = 1 + 763406550x^1
(1, 763406551)
(2, 526813094)
(3, 290219637)
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src> echo "2`n2" | py
thons secret.py
模数 p: 1000000007
请输入参与方 id (1/2/3): 请输入用户2的秘密值 s:
用户2的秘密值的多项式及秘密份额:
f2 = 2 + 126990700x^1
(1, 126990702)
(2, 253981402)
(3, 380972102)
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src> echo "3`n2" | py
thons secret.py
模数 p: 1000000007
请输入参与方 id (1/2/3): 请输入用户3的秘密值 s:
用户3的秘密值的多项式及秘密份额:
f3 = 2 + 570972828x^1
(1, 570972830)
(2, 141945651)
(3, 712918479)

```

图 14: 自动化测试输出（第 1 部分）

```
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src
> echo "1" | python count.py
请输入参与方 id (1/2/3): x=1 求得的秘密份额和为: 461370076
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src
> echo "2" | python count.py
请输入参与方 id (1/2/3): x=2 求得的秘密份额和为: 922740147
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src
> echo "3" | python count.py
请输入参与方 id (1/2/3): x=3 求得的秘密份额和为: 384110211
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\data_security2026\lab3\src
> python avg.py
重构总和  $S = a + b + c = 5$ 
平均值 (直接除法): 1.6666666666666667
3 在模 1000000007 下的逆元: 333333336
平均值 (模逆元运算): 666666673
```

图 15: 自动化测试输出 (第 2 部分)

从图 14 和 15 中可以看到, 全部 6 项测试均标记为 PASS。具体而言:

- 测试 1 验证了基础流程的正确性, $S = 18$ 且 $\text{avg} = 6.0$ 。
- 测试 2 验证了门限恢复的一致性: 无论选取 (d_1, d_2) 、 (d_1, d_3) 还是 (d_2, d_3) , 重构的 S 均为 600, 说明任意 2 份局部和均可正确恢复总和。
- 测试 3 验证了非整除情况下输出为浮点数 1.667, 与理论值 $5/3$ 吻合。
- 测试 4 连续运行了两次完整流程, 两次的份额值完全不同 (因为多项式随机系数不同), 但重构总和均为 158。这既验证了随机多项式不改变最终聚合结果, 又验证了随机性有效保护了单个秘密。
- 测试 5 断言份额不等于原始秘密值, 直观体现了 Shamir 方案的隐私性。
- 测试 6 在大值输入下 (每组数据达到百万级), 总和与平均值仍然正确计算, 说明模数 $p = 1000000007$ 足够容纳这些数据而不发生回绕。

(八) 门限组合额外验证

为了进一步确认 (2,3) 门限方案中「任意 2 份恢复」的性质确实成立, 我们单独验证: 使用 d_1, d_2 重构和使用 d_1, d_3 重构的结果一致性。这一验证已包含在自动化测试 2 中, 从图 14 中可以看到 $S_{12} = S_{13} = S_{23} = 600$, 三者完全一致。

至此, 我们从正确性、门限恢复能力、随机隐私性、边界处理和大数据量五个维度系统验证了本次实验的实现, 所有指标均符合预期。

四、 实验心得与体会

本次实验从实验 3.2 的投票统计方案出发, 将其改造为「三方私有数据平均值计算」场景。在方案设计上, 我们并没有引入新的密码学组件, 而是充分利用了 Shamir 秘密共享的加法同态性质: 将投票值 0/1 替换为任意整数, 其余份额分发、局部求和与拉格朗日重构流程完全保持不变。这种改造方式说明, 课程中讲解的基础秘密共享方案具备良好的场景迁移能力。

在实现层面, 我们将协议拆分为四个职责明确的 Python 脚本, 分别对应函数库、秘密分发、份额聚合与结果重构。这种按协议阶段分割的文件结构使实验流程更清晰, 也便于逐步验证每个阶段的正确性。自动化测试覆盖了基础正确性、门限一致性、随机性和隐私直观验证等场景, 为实现的可靠性提供了系统化的支撑。

通过本次实验，我们对 Shamir 门限秘密共享的构造、加法同态的工作方式以及拉格朗日插值在秘密重构中的具体应用有了更直接的理解。同时，我们也注意到本实验在半诚实模型下能够保护各方的原始输入，但最终暴露的总和与平均值意味着安全目标是「隐藏个体值」而非「隐藏聚合值」——这一边界条件在实际应用中需要结合具体业务场景进行审慎评估。若继续扩展，还可以进一步考虑抗合谋的多方方案或引入安全多方计算中的乘法协议，以支持更丰富的聚合函数。

NIKU